

Repair Cost Estimator — Build Writeup

A mobile-first, offline-first PWA for field repair estimates · single set of static files, no server

It's a cold, empty house with one bar of signal. An acquisition agent has minutes to answer the one question that decides the deal, *what will it cost to fix?*, and any miss comes straight out of margin. This app turns that guess into a structured, room-by-room walkthrough that produces a defensible number **and** a go/no-go before they leave the driveway. A ~20-minute spreadsheet scramble becomes a ~5-minute checklist that exports Excel plus photos on the spot, fully offline.

1 · Most interesting design / UX decision

Modeling the property as a list of removable "rooms," not a fixed set of tabs. The reference app hard-codes one Interior, one Kitchen, and a couple of bathrooms. But the brief is emphatic that "every house is different." So the unit of the UI is a **room instance** the agent adds or removes on the fly (Bathroom 1, Bathroom 2, Bedroom 1, a Living/Common area), each carrying only the repair groups relevant to it. Bedroom and Living are their own room types (decoupled from Interior/General, exactly as the brief asks). The payoff is field speed: the agent's screen always mirrors the house they're standing in, and progress is tracked per group across every room so nothing is silently skipped. Everything else, the always-visible running total, the big quantity steppers, the one-tap "No Action Needed," exists to keep data entry fast with one thumb in a dark, empty house. Per-line and project notes ride into the export too, so the estimate doubles as a scope of work for Spark's in-house construction crew, not just an offer justification.

2 · What's broken or fragile

- **Serial OCR accuracy varies with the photo.** Serial-flagged items (furnace, water heater) run on-device OCR (Tesseract.js, vendored for offline) to auto-read the plate, then let the agent confirm. It's reliable on a clean, square, well-lit plate; a glary or angled shot still needs a manual correction, so the field is always editable and the manual path is the guaranteed fallback, never a dead end.
- **iOS storage eviction.** Data lives in localStorage + IndexedDB. iOS can evict storage for sites that aren't installed to the home screen and go unused for weeks. Installing the PWA (which the app prompts for) makes this far less likely, but there is no cloud backup, by design, since the brief requires it to work with no backend.
- **HEIC photos.** Captures are downscaled to JPEG via canvas, which covers Android and modern iOS; very old iOS HEIC edge cases are untested on real hardware.

3 · Creative addition — the Deal Analyzer (and why)

The brief frames the whole business around one decision: *do the numbers work before we make an offer?* So my creative feature answers exactly that. The **Deal Analyzer** pulls the live repair total from the walkthrough, takes the purchase price, ARV, and any holding/closing costs, and shows **projected profit** with a return-on-cost figure and a color-coded **Good Deal / Thin Margin / Pass** verdict. It turns the estimator from a cost calculator into an acquisition decision tool: the agent can stand in the house and know whether to make the offer. It's reliable, works fully offline, and the result also lands in the exported spreadsheet.

4 · What I'd ship with two more days

- **Equipment-age intelligence from the serial.** Decode the manufacture date out of the OCR'd HVAC/water-heater serial and auto-flag units near end-of-life, turning a photo into a "replace vs. keep" signal during the walk.
- **Project comparison and branded PDF estimate:** a one-page PDF for sellers/partners and a side-by-side compare of two deals.
- **Optional cloud sync** behind a toggle (still offline-first) so a walkthrough can be picked up on another device or by the back office.
- **Square-footage helper:** type the room dimensions once and auto-fill sqft quantities across flooring, paint, and tile.

Role of AI in development

I designed and built this app myself. The architecture, the product decisions, and the core logic are mine. I made the calls that shaped it: modeling the property as removable rooms, storing photos in IndexedDB instead of localStorage, vendoring the libraries so it runs fully offline, and building the Deal Analyzer. I used an AI coding assistant as a tool along the way, to speed up boilerplate, scaffold the Playwright tests, and act as a second pair of eyes when I was debugging, but I drove every decision, wrote and reviewed the code, and tested every feature by hand on my own phone (installing it and running it in airplane mode) before deploying. One example of that judgment: I explored a voice-command layer to cut manual entry, but deliberately left it out because reliable speech-to-text needs a network and would break the brief's fully-offline requirement.

Built for the thing Spark said actually wins: a real walkthrough, not a screenshot. It's fast, reliable, and offline on a phone in an empty house, because that's where the deal is decided.